Write Path in OLTP Workloads

A Hands-On PostgreSQL Workshop

Amit Prabhudesai

rootconf 2026

Tech Lead, Microsoft

About Me

- **Tech Lead, Microsoft**
- `documentdb/documentdb`
- Ex-InMobi (China-engg all-up; Architect, InMobi Ad SDK)
- Worn both hats: Engineering Manager *and* Tech Lead

Slides, exercises, and devcontainer:

`amitprabhudesai/oltp-workloads-pg-workshop`

`amitprabhudesai`

`@amitprabhudesai`

About This Workshop

Database systems as fact-keepers

Durability. Records persist reliably — committed data survives crashes and restarts.

Correctness. Change is constant; interleaved reads and writes must faithfully reflect the intent of every update.

Performance. Neither guarantee is useful with unbounded latency or unsustainable write amplification.

This workshop is about the mechanisms PostgreSQL uses to honour all three.

Module 1 — WAL & Durability

How PostgreSQL guarantees durability without sacrificing throughput.

Module 2 — MVCC & Concurrency

How multiple writers coexist and what the hidden costs are.

🛠 Each module: short theory → hands-on exercises → debrief

devcontainer included — `docker` is enough, no local Postgres needed.

Why PostgreSQL?

- DocumentDB is a PostgreSQL extension
- #1 most-used database in the Stack Overflow Developer Survey — **three years running** (2023, 2024, 2025)
- Well-documented internals: source is readable, WAL format is stable, behaviour is inspectable with plain SQL
- The concepts transfer directly:
  MySQL binlog · CockroachDB Raft log · AWS Aurora redo log — all WAL variants
- Open source: you can *read what actually runs*

Stack Overflow Developer Survey 2025

Technology — Databases:

Database	Usage
PostgreSQL	**49 %**
MySQL	40 %
SQLite	33 %
Redis	28 %

survey.stackoverflow.co/2025

Workshop Structure

Module 1 — WAL & Durability

- WAL write path & record anatomy
- Checkpoints & the recovery horizon
- Full-page images: the checkpoint tax
- WAL Writer & `synchronous_commit` *(optional)*

Module 2 — MVCC & Concurrency

- Isolation anomalies & the MVCC model
- Snapshot isolation: xmin, xmax, visibility
- Lost updates & `SELECT FOR UPDATE`
- VACUUM & bloat

All exercises run in a self-contained devcontainer (PostgreSQL 16 + psql).
Open `modules/0X-.../exercises.sql` and follow along.

Module 1 — WAL & Durability

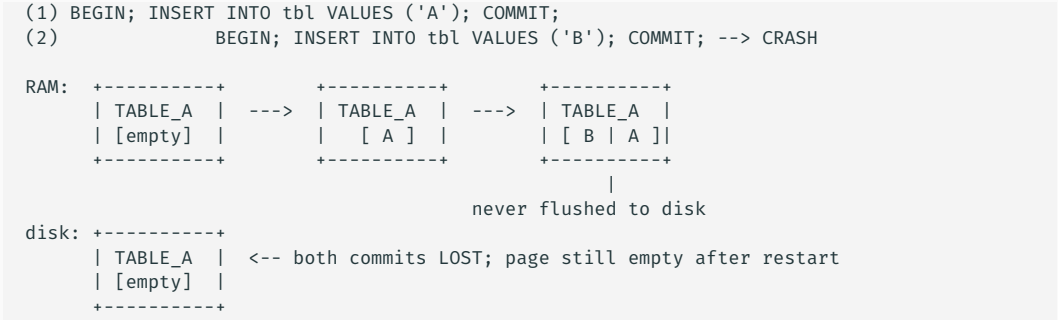
The Durability Problem

A `COMMIT` returns to the client. The modified pages sit in shared buffers[1] — in RAM.

What happens if the server crashes before those pages reach disk?

[1] `https://www.postgresql.org/docs/16/runtime-config-resource.html#GUC-SHARED-BUFFERS`

The Durability Problem



The insight: a *log* is always appended sequentially.[2] If the log is durable, the data pages can be written lazily.

[2] Sequential writes are 10–100× cheaper than random writes on spinning disk; meaningfully cheaper even on SSD.

The Write-Ahead Log

Record all modifications as history data in persistent storage. This history is called **XLOG** or **WAL data**.

Tuple Insertion with WAL

```
INSERT  --> (1) WAL buffer  : XLOG @ LSN_1  (write-ahead)
        --> (2) Shared Buffers: page modified, pd_lsn = LSN_1
COMMIT  --> (3) pg_wal/     : write + fsync  (durable)

INSERT  --> (1) WAL buffer  : XLOG @ LSN_2
        --> (2) Shared Buffers: page modified, pd_lsn = LSN_2
COMMIT  --> (3) pg_wal/     : write + fsync

-----------------------------------------------------

Shared Buffers (RAM)          pg_wal/ (disk)
+----------------------+       +----------------------+
| TABLE_A              |       | XLOG @ LSN_2         |
| [tup_1, tup_2]       |       | XLOG @ LSN_1         |
| pd_lsn = LSN_2       |       | CKPT @ REDO_LSN  (0) |
+----------------------+       +----------------------+
lazy: flushed later             fsynced at COMMIT
(bgwriter / checkpoint)         survives crash
```

Heap tuple layout (header, `xmin`/`xmax`, data): ▸ Appendix A3

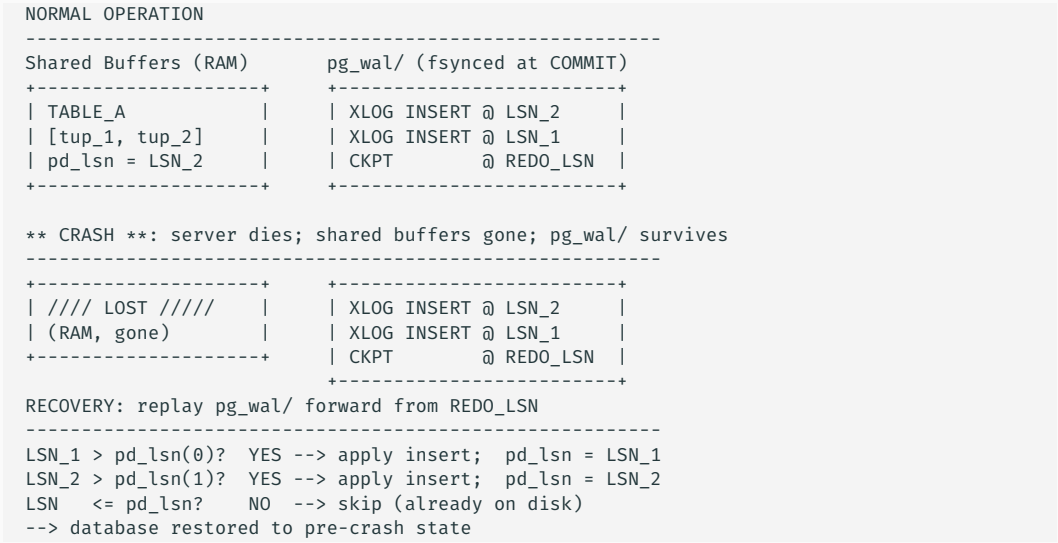
Tuple Insertion with WAL

0. **Checkpoint**: checkpointer writes a checkpoint record containing the latest **REDO point** to WAL.
1. **INSERT**: backend loads the page into shared buffers, inserts the tuple, writes an XLOG record at `LSN_1`, updates the page header: `pd_lsn` `LSN_0` → `LSN_1`.
2. **COMMIT**: WAL buffer flushed to `pg_wal/` (`write` + `fsync`).
3. Subsequent INSERT: XLOG record at `LSN_2`; `pd_lsn` `LSN_1` → `LSN_2`.
4. **CRASH**: shared buffers lost; `pg_wal/` survives on disk. Recovery replays from the REDO point.

exec_simple_query(): WAL Write Path

```
exec_simple_query()                          [postgres.c]
  |
  +-- (1) ExtendCLOG()                       [clog.c]
  |        mark txid IN_PROGRESS in CLOG
  |
  +-- (2) heap_insert()                      [heapam.c]
  |        insert tuple into shared buffer page
  |   |
  |   +-- (3) XLogInsert()                   [xloginsert.c]
  |            write Heap/INSERT record to WAL buffer
  |            update page pd_lsn := LSN_1
  |
  +-- (4) finish_xact_command()              [postgres.c]
      |    commit action
      |
      +-- (4a) XLogInsert()                  [xloginsert.c]
      |         write Commit record to WAL buffer (LSN_2)
      |
      +-- (5) XLogWrite()                    [xlog.c]
      |        flush WAL buffer -> WAL segment file
      |        (fsync / fdatasync / O_DSYNC per wal_sync_method)
      |
      +-- (6) TransactionIdCommitTree()      [transam.c]
               mark txid COMMITTED in CLOG
```

Database Recovery Using WAL

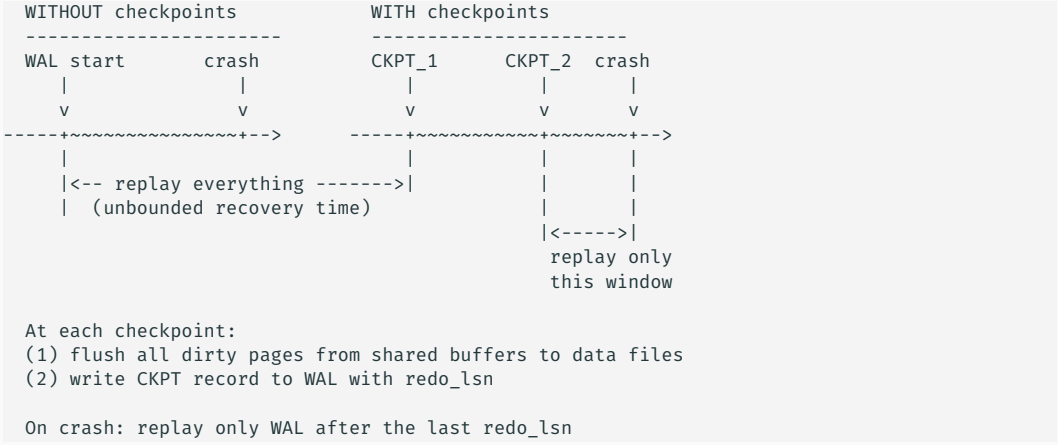


Checkpoints

Recovery replays WAL from some starting point. If that starting point is the very beginning of WAL history, recovery time is unbounded.

How does the database keep restart time predictable?

Checkpoints: Bounding Recovery Time



Triggered by: `checkpoint_timeout` (default 5 min) or `max_wal_size` exceeded. Larger `max_wal_size` = better throughput, longer recovery.

Exercise 1.3 — Checkpoints & Write Path

`>_ modules/01-wal-durability/exercises.sql`

Run an explicit `CHECKPOINT`, insert rows, watch the recovery horizon grow in `pg_stat_wal`.

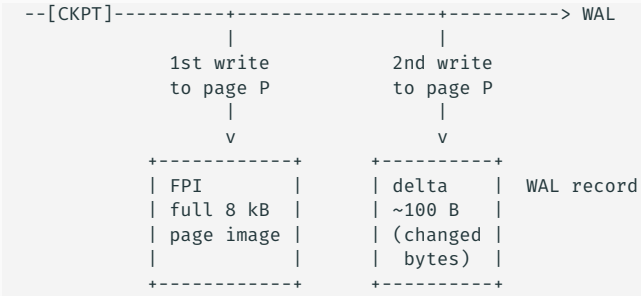
Checkpoint again; check `pg_stat_bgwriter` for `buffers_backend` — the write-pressure signal.

Full-Page Writes

WAL records the *change* — a delta. Recovery replays that delta onto the page on disk.

But what if the page on disk was only half-written when the crash happened?

Full-Page Writes: Backup Blocks



Full-Page Images: The Checkpoint Tax

After each checkpoint, the first write to any dirty page embeds the *entire* 8 kB page in the WAL record — not just the delta. This guarantees recovery even from a torn page.

There are no free lunches. What does this cost?

Exercise 1.1 — Observing WAL Generation

`>_ modules/01-wal-durability/exercises.sql`

Snapshot `pg_current_wal_lsn()`, insert rows, measure the delta.

Force a `CHECKPOINT`, update rows, watch `wal_fpi` spike in `pg_stat_wal`.

WAL Performance Tuning

WAL Performance Tuning

Optional — cover if time allows

- **The WAL Writer & group commit** — how PostgreSQL amortises `fsync()` cost across concurrent transactions
- **`synchronous_commit`** — the per-transaction knob for trading a small durability window for throughput

The WAL Writer Process

Every committed transaction must flush WAL to disk.

With hundreds of concurrent commits per second — how do you avoid turning that into hundreds of sequential `fsync()` calls?

The WAL Writer Process: Group Commit

What it does:

- Background process; wakes every `wal_writer_delay` (default 200 ms)
- Flushes pending WAL from shared buffers to `pg_wal/`
- Also woken immediately by backends waiting on `synchronous_commit = on`

Group commit:

- Multiple concurrent `COMMIT`s accumulated in the WAL buffer are flushed in a single `write()` + `fsync()`
- With high concurrency: `wal_write` $\ll$ number of commits — a real saving on spinning disk, meaningful on NVMe

Observable in `pg_stat_wal`: compare `wal_write` and `wal_sync` counts against the commit count.

The catch: each committing backend still *waits* for the shared fsync to complete before `COMMIT` returns — the cost is amortized across concurrent transactions, not eliminated.

Durability vs Performance tradeoff

Durability has a cost: every `COMMIT` waits for WAL to be fsynced to disk before returning to the client.

Can the database offer a tunable knob for trading off RPO[3] and performance for different kinds of workloads?

[3] RPO (Recovery Point Objective) — the maximum committed data loss a workload can tolerate after a crash.

`synchronous_commit`: The Durability Dial

Set per-session or per-transaction. Cluster default: **`on`**.

Setting	COMMIT returns when …	Max data loss
`on`	WAL written *and* fsynced locally	none
`remote_apply`	WAL applied on standby before COMMIT returns	none on primary
`remote_write`	WAL written to standby OS buffer	standby hard crash
`local`	WAL fsynced locally only (standby may lag)	none on primary
`off`	WAL appended to shared buffer	$\leq$ `wal_writer_delay`

Key: `off` does *not* risk corruption. The database always stays **consistent** — you only risk losing the last ≤ 200 ms of committed data.

Exercise 1.2 — synchronous_commit

>_ modules/01-wal-durability/exercises.sql

Run the benchmark. Watch `wal_write` and `wal_sync` in `pg_stat_wal`.

The sync *count* is the signal — not the wall-clock time.

Module 1: Hands-On Exercises

Exercise 1.1 — Observing WAL generation

- Snapshot LSN with `pg_current_wal_lsn()`
- Insert rows; measure WAL produced
- Force a checkpoint; watch `wal_fpi` spike

Exercise 1.3 — Checkpoints & write path

- Establish baseline with explicit `CHECKPOINT`
- Insert rows; watch recovery horizon grow
- Checkpoint; watch it shrink
- Read `pg_stat_bgwriter` for I/O accounting

Exercise 1.2 — `synchronous_commit` *(optional)*

- Run inserts with `on` vs `off`
- Compare `wal_write` and `wal_sync` deltas
- Understand why the sync count is the real signal

`>_ modules/01-wal-durability/exercises.sql`

Module 2 — MVCC & Concurrency

The Isolation Problem

```
Initial:  Arun=1000  Babita=500  Chirag=1500  (total = 3000)
-------------------------------------------------------------

T1: transfer Arun -> Babita (300)    T2: audit (read all balances)
---------------------------------   ---------------------------
BEGIN
UPDATE Arun   1000 -> 700
UPDATE Babita  500 -> 800
                                     BEGIN
                                     SELECT Arun   --> 1000 (*)
COMMIT <---- commits here ----------+
                                     SELECT Babita -->  800 (**)
                                     SELECT Chirag --> 1500
                                     -------------------------
                                     total = 3300  (?!)
-------------------------------------------------------------
(*) T1 not yet committed: T2 sees Arun = 1000
(**) T1 already committed: T2 now sees Babita = 800

T2 observed both the old Arun and the new Babita.
300 appeared out of thin air.
```

Approach 1: Locking

```
T1: transfer Arun -> Babita (300)    T2: audit
---------------------------------    ------------------------
BEGIN
UPDATE Arun   1000 -> 700
UPDATE Babita  500 -> 800
(holds locks on both rows)           BEGIN
                                     SELECT Arun ... BLOCKED
                                         |
                                         | waiting for T1
                                         |
COMMIT  (releases locks) ----------->   |
                                     SELECT Arun   --> 700
                                     SELECT Babita --> 800
                                     SELECT Chirag --> 1500
                                     ------------------------
                                     total = 3000  (correct)
```

Correct — but T2 sat idle while T1 worked. At high concurrency, that wait queue becomes the bottleneck.

Approach 2: MVCC

```
T1: transfer Arun -> Babita (300)    T2: audit
---------------------------------    ------------------------
BEGIN
UPDATE Arun   1000 -> 700            BEGIN  <-- snapshot here
UPDATE Babita  500 -> 800
                                     SELECT Arun   --> 1000 (*)
COMMIT
                                     SELECT Babita -->  500 (*)
                                     SELECT Chirag --> 1500
                                     ------------------------
                                     total = 3000  (correct)

(*) T2 reads the row version visible at its snapshot time.
    T1 and T2 never blocked each other.
```

How does PostgreSQL know which version to show T2? → next.

MVCC: Multi-Version Concurrency Control

In a lock-based system, a writer holds an exclusive lock on a row while it works. Every concurrent reader must wait.

At high concurrency, that wait queue becomes the bottleneck. Can we do better?

MVCC: The Core Idea

```
accounts heap page, after T1 (transfer) commits:

+-------------------------------------------------+
| tuple_1: xmin=T0  xmax=T1  Arun  balance=1000   | <- old
| tuple_2: xmin=T1  xmax=0   Arun  balance= 700   | <- live
+-------------------------------------------------+
T0 = txid that originally set Arun's balance to 1000

T2 snapshot taken before T1 committed:
  reads tuple_1 --> Arun = 1000

T3 snapshot taken after T1 committed:
  reads tuple_2 --> Arun = 700

Both reads served from the same page.
Neither T2 nor T3 blocked T1.
```

MVCC: The Guarantee

Readers never block writers. Writers never block readers.

Each transaction sees a consistent snapshot of the database as it existed when the transaction began.

Tuple Versions: `xmin` and `xmax`

```
SELECT xmin, xmax, ctid, name, balance FROM accounts WHERE name = 'Arun';

Before T1:
 xmin | xmax | ctid  | name  | balance
------+------+-------+-------+---------
  100 |    0 | (0,1) | Arun  |    1000

After T1 (txid 200) commits:
 xmin | xmax | ctid  | name  | balance
------+------+-------+-------+---------
  100 |  200 | (0,2) | Arun  |    1000   <- xmax=200: superseded by T1
  200 |    0 | (0,2) | Arun  |     700   <- live
```

`xmin`: txid that created this version `xmax`: txid that superseded it (`0` = live) `ctid`: physical location of the live version.

Transaction Snapshots

Multiple versions of the same row coexist on disk. `xmin` and `xmax` say who created them.

But how does a transaction decide which version it is allowed to see?

Transaction Snapshots: The Visibility Fence

```
Snapshot format:  xmin : xmax : xip_list   (pg_current_snapshot())

(a) "100:100:"    no transactions in flight at snapshot time

 past <--[...][98][99](100)(101)(102)(103)(104)[...]--> future
                    ^
                  xmin=xmax=100
 [n] = visible     (n) = invisible (active or not yet started)

(b) "100:104:100,102"    txids 100 and 102 still in flight

 past <--[...][98][99](100)[101](102)[103](104)(105)[...]--> future

 98,99:      committed before xmin         -> visible
 100,102:    in xip_list (in-progress)     -> invisible
 101,103:    committed, not in xip_list    -> visible
 104,105:    >= xmax, not yet started      -> invisible
```

Transaction Snapshots: When is Yours Taken?

```
time   T1: transfer (txid 200)   T2: READ COMMITTED    T3: REPEATABLE READ
----   ----------------------    ------------------    -------------------
 t1    BEGIN
       UPDATE Arun 1000->700     BEGIN                 BEGIN
 t2                              SELECT Arun           SELECT Arun
                                 snapshot @ t2         snapshot @ t2
                                 --> sees 1000         --> sees 1000
 t3    COMMIT
 t4                              SELECT Arun           SELECT Arun
                                 NEW snapshot @ t4     SAME snapshot (t2)
                                 --> sees 700          --> sees 1000
```

`READ COMMITTED`: fresh snapshot per statement — sees T1's commit at t4.

`REPEATABLE READ`: snapshot fixed at `BEGIN` — consistent view throughout.

Visibility Check Rules

```
For each tuple, test xmin and xmax against the current snapshot:

t_xmin ABORTED?               --> invisible  (rolled back)

t_xmin IN_PROGRESS?
  same txid, t_xmax = 0       --> visible    (my own insert)
  otherwise                   --> invisible  (dirty read blocked)

t_xmin COMMITTED?
  t_xmin active in snapshot   --> invisible  (snapshot fence)
  t_xmax = 0 or ABORTED       --> visible    (live tuple)
  t_xmax IN_PROGRESS != me    --> visible    (concurrent delete pending)
  t_xmax COMMITTED,
    not active in snapshot     --> invisible  (deleted and visible)
```

Dirty reads are structurally impossible: a tuple written by an in-progress transaction is always invisible to other sessions.

The Lost Update Problem

MVCC guarantees readers never block writers and vice versa.

But what about two transactions that both read the same value and then update it? Does MVCC protect against that?

The Lost Update Problem

```
Initial:  Arun=1000  Babita=500  Chirag=1500  (total = 3000)
-------------------------------------------------------------
T1: Arun -> Babita (300)          T2: Chirag -> Arun (200)
--------------------------------  --------------------------------
BEGIN
SELECT Arun --> 1000
                                  BEGIN
                                  SELECT Arun --> 1000   (*)
new_balance = 1000 - 300 = 700
UPDATE Arun SET balance = 700
COMMIT
                                  new_balance = 1000 + 200 = 1200
                                  UPDATE Arun SET balance = 1200
                                  COMMIT
-------------------------------------------------------------
(*) T2 read Arun before T1 committed.  Computed from stale data.
Final:    Arun = 1200   Expected: Arun = 900
T1's debit of 300 is gone.
```

SELECT FOR UPDATE

A relative **UPDATE** is safe for simple arithmetic. But some workflows must read a value first — check a limit, compute a fee, apply business logic — before writing.

How do you make that read-then-write sequence atomic under concurrent writers?

SELECT FOR UPDATE

```
Initial:  Arun=1000  Babita=500  Chirag=1500
-------------------------------------------------------------
T1: Arun -> Babita (300)          T2: Chirag -> Arun (200)
--------------------------------  ------------------------------
BEGIN
SELECT Arun FOR UPDATE            BEGIN
--> ExclusiveLock acquired        SELECT Arun FOR UPDATE
--> Arun = 1000                   --> BLOCKS  (lock held by T1)
new_balance = 1000 - 300 = 700         |
UPDATE Arun SET balance = 700          |
COMMIT  --> lock released              |
                                  --> unblocks
                                  --> Arun = 700   (*)
                                  new_balance = 700 + 200 = 900
                                  UPDATE Arun SET balance = 900
                                  COMMIT
-------------------------------------------------------------
(*) T2 re-reads the post-commit value.  No stale data.
Final:    Arun = 900   Expected: Arun = 900   OK
```

SELECT FOR UPDATE: How It Works

Row-level ExclusiveLock

`SELECT … FOR UPDATE` acquires a row-level **ExclusiveLock** *before* returning the value to the application. A concurrent session attempting the same lock blocks until the holder commits or rolls back.

REPEATABLE READ: first-updater-wins

Under `REPEATABLE READ`, the waiting session does *not* silently overwrite. On unblock it detects the concurrent update and aborts with a serialization error. The application must catch and retry.

Inspect live locks with `pg_locks` and `pg_blocking_pids()` — Exercise 2.4.

Module 2: Hands-On Exercises

Exercise 2.1 — Snapshot isolation

- INSERT in Session A, don't commit
- Session B cannot see it
- Commit; Session B's next SELECT sees it

Exercise 2.2 — Isolation levels

- Session A in REPEATABLE READ: snapshot fixed
- Session B inserts and commits
- Session A re-counts: same result
- Repeat with READ COMMITTED: different result

Exercise 2.3 — Lost update

- Race two sessions on the same row
- Compare relative vs application read-modify-write
- Observe which debit survives

Exercise 2.4 — `SELECT FOR UPDATE`

- Session A locks a row; Session B blocks
- Inspect `pg_locks` and `pg_blocking_pids()`
- Session A commits; B unblocks on the post-commit value

`>_ modules/02-mvcc-concurrency/session_{a,b}.sql`

VACUUM

Every `UPDATE` leaves a dead tuple on the heap page. Every `DELETE` leaves one too.

Dead tuples accumulate. Concurrent readers may still need an old version, so the database cannot reclaim it immediately.

Who cleans this up — and when?

VACUUM

Autovacuum runs in the background, scanning tables periodically and reclaiming space occupied by dead tuples that no active transaction can see any more.

VACUUM: Overview

What VACUUM does:

- Marks dead tuple slots as reusable (does *not* shrink the file)
- Updates the **visibility map** — pages with no dead tuples are flagged and skipped on the next pass
- **Freezes** old tuples, replacing their `xmin` with a special frozen ID to prevent transaction ID wraparound

Key signals:

- `pg_stat_user_tables.n_dead_tup` — dead tuple backlog
- `age(datfrozenxid)` in `pg_database` — distance from XID wraparound

```
After many UPDATEs:

heap page
+---------------------------+
| dead | dead | live | live |
|  tup |  tup |  tup |  tup |
+---------------------------+
  ^      ^
  VACUUM reclaims these slots;
  space is reused for new tuples

Without VACUUM:
  page fills with dead tuples
  --> new rows spill to new pages
  --> table grows (bloat)
  --> sequential scans slow down
```

Takeaways

Key Takeaways: WAL & Durability

WAL & Durability

- WAL is why PostgreSQL survives crashes: *always write the log before the data page*
- `synchronous_commit` is the highest-leverage throughput knob; measure sync *counts*, not just timing
- Checkpoints bound recovery time; tune `max_wal_size` to balance throughput vs restart latency
- Full-page images are unavoidable; reduce overhead with `wal_compression`

Key Takeaways: MVCC & Concurrency

MVCC & Concurrency

- MVCC prevents isolation anomalies without locking — each transaction sees a consistent snapshot; readers never block writers
- Isolation level controls snapshot timing — READ COMMITTED re-snapshots per statement; REPEATABLE READ holds one
- Lost updates are *not* prevented by MVCC: use relative `UPDATE` or `SELECT FOR UPDATE`
- Dead tuples from every `UPDATE`/`DELETE` must be reclaimed by VACUUM — left unchecked, this becomes table bloat

Thank You!

Questions? Let's dig in.

 amitprabhudesai/oltp-workloads-pg-workshop

Exercises, devcontainer, and these slides are all in the repo.

 prabhudesai.amit@gmail.com

References

- Hironobu Suzuki, *The Internals of PostgreSQL*, `https://www.interdb.jp/pg/`. Chapter 5 (Concurrency Control) and Chapter 9 (WAL). © All Rights Reserved, Hironobu Suzuki. Used for noncommercial educational purposes per the author's terms; see `https://www.interdb.jp/pg/` for licence details.
- PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, `https://www.postgresql.org/docs/16/`. Particularly: *Reliability and the Write-Ahead Log* and *WAL Configuration*.
- Stack Overflow, *Developer Survey 2025 — Technology*, `https://survey.stackoverflow.co/2025/technology`. PostgreSQL #1 most-used database, three years running (2023–2025).

Bonus: Write Amplification

One UPDATE. How Many WAL Records?

MVCC means every **UPDATE** is a new heap tuple. When the updated column is indexed, the index must change too.

Two updates. Same table. Same row. How much WAL does each generate?

```
-- A: balance is not indexed
UPDATE accounts SET balance = balance - 100.00 WHERE id = 42;

-- B: owner is indexed
UPDATE accounts SET owner = 'alice_b' WHERE id = 42;
```

One of these generates 3–4× more WAL than the other. Which one, and why?

HOT Updates: Skipping the Index WAL

```
-- A: balance not indexed -> HOT update
-- Heap: one HOT_UPDATE record (~150 bytes)
-- Btree: nothing

-- B: owner indexed -> must update the index too
-- Heap: one UPDATE record
-- Btree: DELETE old entry + INSERT new entry (~2 records, ~600 bytes total)
```

HOT (Heap Only Tuple) fires when two conditions both hold:

1. No indexed column was changed
2. The new tuple fits on the *same heap page* as the old one

Verify with `pg_walinspect`:

- `resource_manager = 'Btree'` records: 0 for HOT, 2 for indexed update
- `record_type = 'HOT_UPDATE'` vs `'UPDATE'` in the Heap record

Lever 1: `fillfactor`

HOT condition 2 — new tuple fits on the same page — fails when the page is full. `fillfactor` reserves space so updates stay in-place.

```
-- Default: fillfactor = 100 (pages packed full; HOT degrades as table grows)
-- Better for update-heavy tables:
CREATE TABLE accounts (...) WITH (fillfactor = 70);
-- or on an existing table:
ALTER TABLE accounts SET (fillfactor = 70);
VACUUM accounts;   -- rewrites pages at the new fill target
```

- 70–80 is a common starting point for tables with frequent UPDATEs on non-indexed columns
- Trade-off: larger table footprint on disk, fewer pages per scan
- Confirm HOT is firing: `pg_stat_user_tables.n_tup_hot_upd` should be a large fraction of `n_tup_upd`

Lever 2: Index Audit

Every index on a column that gets `UPDATE`d forces non-HOT writes and Btree WAL — whether or not the index is ever used by a query.

```
-- Find indexes with zero scans since last stats reset
SELECT schemaname, relname, indexrelname, idx_scan
FROM   pg_stat_user_indexes
WHERE  idx_scan = 0
ORDER  BY schemaname, relname;
```

- `idx_scan = 0` is a candidate for removal — but check `pg_stat_reset()` history first
- Partial indexes (`WHERE status = 'pending'`) cover fewer rows and are HOT-compatible for rows outside the partial condition
- Before dropping: `pg_get_wal_records_info()` to confirm the index is contributing Btree WAL on your hottest `UPDATE` path

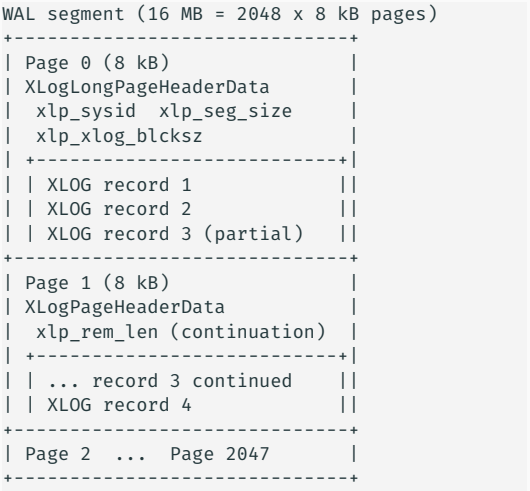
Appendix

PostgreSQL internals — deeper cuts

A1. Internal Layout of a WAL Segment

A WAL segment is **16 MB** by default, subdivided into **8 kB pages**:

- **Page 0**: `XLogLongPageHeaderData` — includes `xlp_sysid`, segment size, block size; used as a cross-check on open
- **Pages 1+**: `XLogPageHeaderData` — includes `xlp_magic`, `xlp_tli`, `xlp_pageaddr`, `xlp_rem_len`
- XLOG records written sequentially; a record may *span* page boundaries (`xlp_rem_len` tracks the continuation length)



A2. Internal Layout of an XLOG Record

Since PostgreSQL 9.5, XLOG records use a common structured format.

Header (`XLogRecord`):

- `xl_tot_len` — total record length
- `xl_xid` — transaction ID
- `xl_prev` — LSN of previous record
- `xl_rmid` / `xl_info` — resource manager and subtype (e.g. `RM_HEAP` + `XLOG_HEAP_INSERT`)
- `xl_crc` — integrity check

Recovery dispatches to the right replay function (`heap_xlog_insert`, `heap_xlog_update`, …) via `xl_rmid` + `xl_info`.

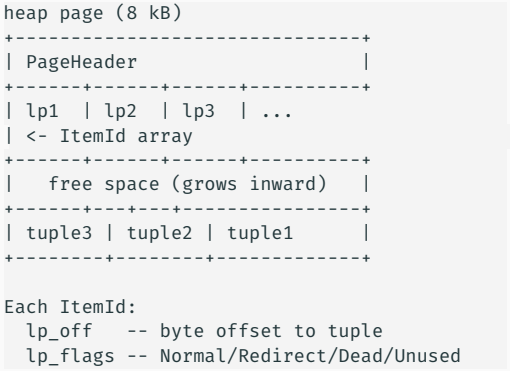
```
Non-backup block (INSERT delta):
+-----------------------+
| XLogRecord (header)   |
|  xl_rmid=RM_HEAP      |
|  xl_info=HEAP_INSERT  |
+-----------------------+
| XLogRecordBlockHeader |
|  relfilenode, fork,   |
|  block_no, data_len   |
+-----------------------+
| XLogRecordDataHeader  |
+-----------------------+
| block data (delta)    |
+-----------------------+
| xl_heap_insert (main) |
+-----------------------+

Backup block: block data replaced by
full 8 kB page (FPI); adds
XLogRecordBlockImageHeader with
length, hole_offset, bimg_info.
```

A3. Tuple Structure

Each row version on a heap page is a **HeapTuple**:

- **`t_xmin`** — inserting transaction ID
- **`t_xmax`** — deleting/updating transaction ID (0 = live)
- **`t_ctid`** — pointer to the newer version of this row (self-referential if this is the latest)
- **`t_infomask`** — hint bits for commit status (avoids repeated clog lookups)
- **`t_hoff`** — offset to user data



`t_ctid` forms the version chain: old tuple points to new. VACUUM marks dead ItemIds as **`Unused`** and reclaims their space.

◂ Back

A4. Buffer Manager Structure

Three layers between a backend and a page on disk:

Key descriptor fields:

- **tag** — `(relfilenode, fork, block_number)`
- **dirty** — modified; must reach disk before slot is reused
- **refcount** — pin count; `refcnt > 0` blocks eviction
- **usage_count** — access frequency; drives **clock-sweep**
- **freelist** — empty descriptors chained at startup

`shared_buffers`: default 128 MB; typically 25 % of RAM in production.

A5. INSERT, DELETE, UPDATE on the Heap

```
INSERT:
  tuple_1: xmin=100 xmax=0   ctid=(0,1)  name='Jekyll'
  -- one new tuple, no dead tuple

DELETE (by txid 200):
  tuple_1: xmin=100 xmax=200 ctid=(0,1)  name='Jekyll'
  -- tuple_1 is now "dead" once txid 200 commits
  -- still physically on disk until VACUUM

UPDATE (by txid 201):
  tuple_1: xmin=100 xmax=201 ctid=(0,2)  name='Jekyll'  <- dead
  tuple_2: xmin=201 xmax=0   ctid=(0,2)  name='Hyde'    <- live
  -- UPDATE = logical DELETE + INSERT
  -- Every UPDATE creates one dead tuple
  -- Hot heavy-update tables bloat without VACUUM
```

`t_ctid` in tuple_1 points to tuple_2, so index scans can follow the chain to the live version without re-scanning. This is the basis of **HOT** (Heap Only Tuple) updates.

A6. exec_simple_query(): WAL Write Path

```
exec_simple_query()                          [postgres.c]
  |
  +-- (1) ExtendCLOG()                       [clog.c]
  |       mark txid IN_PROGRESS in CLOG
  |
  +-- (2) heap_insert()                      [heapam.c]
  |       insert tuple into shared buffer page
  |   |
  |   +-- (3) XLogInsert()                   [xloginsert.c]
  |           write Heap/INSERT record to WAL buffer
  |           update page pd_lsn := LSN_1
  |
  +-- (4) finish_xact_command()              [postgres.c]
      |    commit action
      |
      +-- (4a) XLogInsert()                  [xloginsert.c]
      |        write Commit record to WAL buffer (LSN_2)
      |
      +-- (5) XLogWrite()                    [xlog.c]
      |       flush WAL buffer -> WAL segment file
      |       (fsync / fdatasync / O_DSYNC per wal_sync_method)
      |
      +-- (6) TransactionIdCommitTree()      [transam.c]
              mark txid COMMITTED in CLOG
```

A7. WAL as a Change Stream: Logical Decoding

How it works

- `wal_level = logical` records enough context to reconstruct row-level changes
- A **replication slot** pins WAL until the consumer ACKs each LSN — guarantees no gaps
- Output plugins decode raw XLOG records into a change stream: `test_decoding` (built-in), `wal2json`, `pgoutput` (logical replication)

Production uses

- CDC pipelines — Debezium, Airbyte, Fivetran
- Audit logs without triggers
- Live zero-downtime migrations
- Event sourcing / outbox pattern

Operational gotcha: an unconsumed slot holds WAL indefinitely. Monitor `pg_replication_slots.wal_status` and set `max_slot_wal_keep_size` to bound disk exposure.

A7b. Logical Decoding: Reading the Change Stream

```
-- One-time setup (superuser)
SELECT pg_create_logical_replication_slot('audit', 'test_decoding');

-- peek: non-destructive; get: advances the slot
SELECT lsn, xid, data
FROM pg_logical_slot_peek_changes('audit', NULL, NULL);

 lsn       | xid  | data
-----------+------+------------------------------------------------------------
 0/9368D48 | 1001 | BEGIN 1001
 0/9368DB0 | 1001 | table rootconf.accounts: INSERT: id[bigint]:264
           |      |   owner[text]:'alice'  balance[numeric]:100.00
 0/936BB80 | 1001 | COMMIT 1001
 0/936BB80 | 1002 | BEGIN 1002
 0/936BB80 | 1002 | table rootconf.accounts: UPDATE: id[bigint]:264
           |      |   balance[numeric]:200.00
 0/936BC18 | 1002 | COMMIT 1002
```

Each decoded event carries the LSN, transaction boundary, and full column values — exactly what a CDC consumer ingests.

A8. Commit Log (clog)

What is clog?

- Stores the *commit status* of every transaction: IN_PROGRESS / COMMITTED / ABORTED / SUB_COMMITTED
- Physical location: `$PGDATA/pg_xact/`
- 2 bits per transaction; 256 kB file covers 1M transactions
- Cached in shared memory (`pg_xact` buffer pool)
- Visibility checks consult clog to evaluate `t_xmin`/`t_xmax` status

```
pg_xact/0000  (first file)
  txid 0: IN_PROGRESS
  txid 1: COMMITTED
  txid 2: ABORTED
  ...

Each 2-bit entry:
  00 = IN_PROGRESS
  01 = COMMITTED
  10 = ABORTED
  11 = SUB_COMMITTED
```

Old clog files removed by VACUUM once all transactions referencing them are frozen.

Hint bits (`t_infomask`) cache the result of clog lookups directly in the tuple header, so subsequent reads skip the clog entirely for already-settled transactions.

A9. VACUUM: Dead Tuples, Bloat, and Wraparound

Why VACUUM exists:

- Every `UPDATE`/`DELETE` leaves a *dead tuple* on disk — MVCC cannot reclaim it immediately
- Concurrent readers might still need the old version
- **Autovacuum** (default: on) scans tables periodically and reclaims dead tuple space; `pg_stat_user_tables.n_dead_tup` shows the backlog
- Unchecked: pages fill with dead tuples, table grows (*bloat*), sequential scans slow down

Transaction ID wraparound:

- txid is a 32-bit counter: wraps at ∼4 billion
- PostgreSQL uses modular arithmetic with a 2^{31} horizon: tuples with `xmin` older than 2^{31} become *in the future* — a catastrophic visibility inversion
- Fix: VACUUM *freezes* old tuples, replacing their `xmin` with a special frozen txid that is always visible

```sql
-- How far are we from wraparound?
SELECT datname,
  age(datfrozenxid) AS xid_age
FROM pg_database
ORDER BY xid_age DESC;
```

Autovacuum warns at 200 M transactions from wraparound; PostgreSQL forces a shutdown at 3 M to